

Quantization, Deeper

Outliers, groups, and better grids · video 2:30

This lesson answers one question: why does naive 4-bit rounding hurt a model, and how do real tools make it almost free? The culprit is **outliers**: a few large values set the scale and crush everything else. Giving every **group** of 32 to 128 weights its own scale fixes most of the damage for a fraction of a bit; the **NF4** grid places its levels where weights actually are; and for activations, where the outlier is a whole token (the attention sink), one scale **per token** makes 8-bit inputs nearly free.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Outliers set the scale	▶ 0:08-0:35	2	☐
2. Groups and their cost	▶ 0:35-1:12	2	☐
3. A better grid: NF4	▶ 1:12-1:32	1	☐
4. Activations: the outlier is a token	▶ 1:32-2:11	2	☐

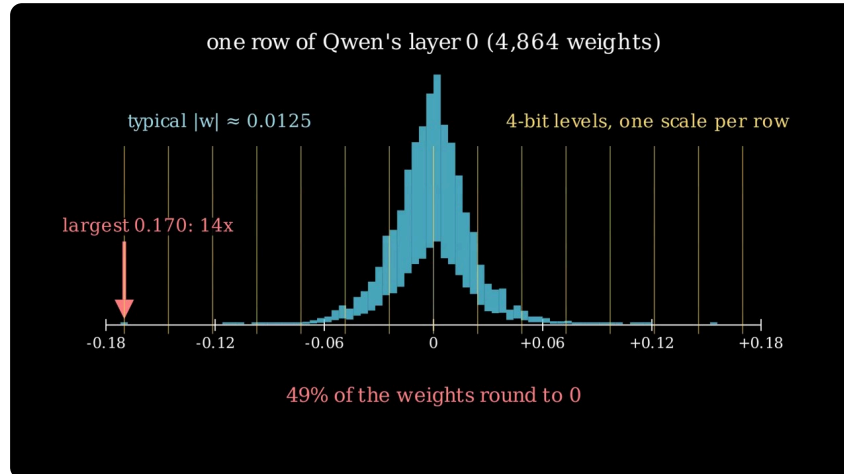
Before you start: LLMs in Practice episode 10 (quantization basics) and Deep Dive episode 12 (attention sinks). Code: `code/d33_quantization` (Qwen2.5-0.5B, about 1 GB).

CONCEPT 1 OF 4

Outliers set the scale

▶ Video 0:08-0:35

Symmetric 4-bit rounding uses the integers -7 to 7 : the step is the row's largest absolute value divided by 7 . Rounding to 4 bits makes a 32-bit model 8 times smaller. Done with one scale per row, it also makes it much worse.



▶ 0:34 One row of layer 0's down_proj: typical $|w|$ 0.0125, largest 0.170. Four bits with one scale give steps of 0.024: 49% of the weights round to 0.

The reason is outliers. In one row of Qwen2.5-0.5B's first layer the typical (median) weight is 0.0125 and the largest 0.170, 14 times bigger. The step becomes 0.024, so every weight smaller than half a step, 49% of them, rounds to exactly zero. Across all 304,128 rows, the largest weight is typically 6 times the median, and up to 145 times.

KEY IDEA

One large value sets the scale for everything that shares it.

Check 1: Outliers set the scale

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

A row's largest absolute weight is 0.35. With symmetric 4-bit rounding ($-7..7$), what is the step, and which weights round to 0?

1.2 CALCULATION

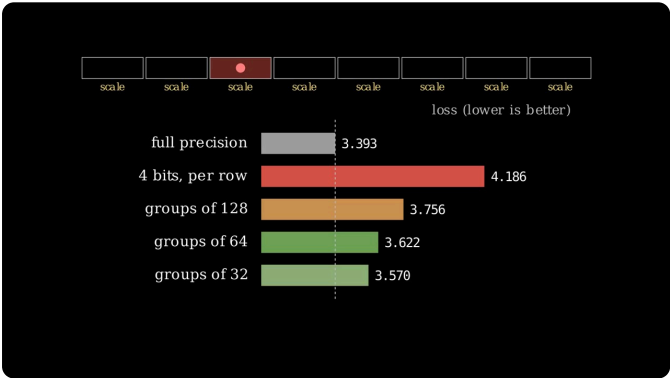
How many levels does symmetric 3-bit rounding use?

Groups and their cost

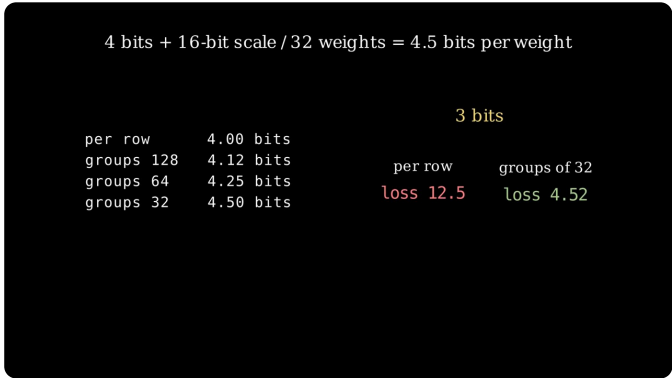
▶ Video 0:35-1:12

The fix is groups: every block of, say, 64 consecutive weights gets its own scale, so an outlier only spoils its own group. Mean loss on Shakespeare, with every linear layer's weights rounded to 4 bits:

4-BIT WEIGHTS	LOSS	WEIGHT ERROR	BITS PER WEIGHT
full precision	3.393	-	32
one scale per row	4.186	17.9%	4.00
groups of 128	3.756	13.2%	4.12
groups of 64	3.622	11.8%	4.25
groups of 32	3.570	10.4%	4.50



▶ 0:55 Loss with 4-bit weights: full precision 3.393; per row 4.186; groups of 128, 64, 32: 3.756, 3.622, 3.570.



▶ 1:10 Scales cost 16 / group bits per weight; at 3 bits, groups of 32 rescue the model (12.5 → 4.52).

The scales aren't free: one 16-bit scale per 32 weights adds half a bit per weight. It is worth it, and more so at lower precision: at 3 bits, one scale per row destroys the model (loss 12.5), and groups of 32 bring it back to 4.52.

KEY IDEA
Smaller groups isolate outliers; each scale costs 16 / group bits per weight.

Check 2: Groups and their cost
Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION
How many gigabytes does a 7-billion-parameter model take at 4 bits with one 16-bit scale per 32 weights? And in 16 bits?

2.2 HANDS-ON CODE

Try it yourself. Round this row to 4 bits with one scale, then with groups of 4. Which weights survive?

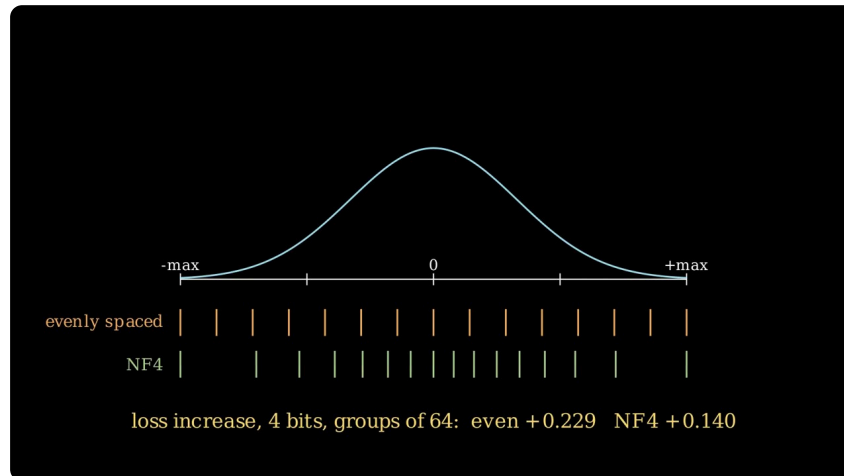
```
import torch
w = torch.tensor([[0.01, -0.02, 0.015, 0.9, 0.012, -0.008, 0.02, -0.011]])
def quant(w, g):
    x = w.reshape(1, -1, g)
    s = x.abs().amax(-1, True) / 7
    return ((x / s).round() * s).reshape(w.shape)
print(quant(w, 8)); print(quant(w, 4))
```

CONCEPT 3 OF 4

A better grid: NF4

▶ Video 1:12-1:32

Weights follow a bell curve: most are near zero. Evenly spaced levels waste steps on the rare large values. **NF4** (from QLoRA) places its 16 levels at quantiles of a normal distribution, so each level covers about the same share of weights: $-1.00, -0.70, -0.53, \dots, 0, \dots, 0.56, 0.72, 1.00$, times the group's scale. Same 4 bits; the loss increase over full precision drops from **+0.229** to **+0.140**.



▶ 1:31 Evenly spaced levels vs NF4 levels, denser near zero. Loss increase at 4 bits, groups of 64: +0.229 vs +0.140.

KEY IDEA

Put the levels where the values are.

Check 3: A better grid: NF4

Answer all 1 questions before looking at the answers. Pass mark: all correct.

3.1 TRUE OR FALSE

“NF4 is better than evenly spaced 4-bit levels because it stores more bits.”

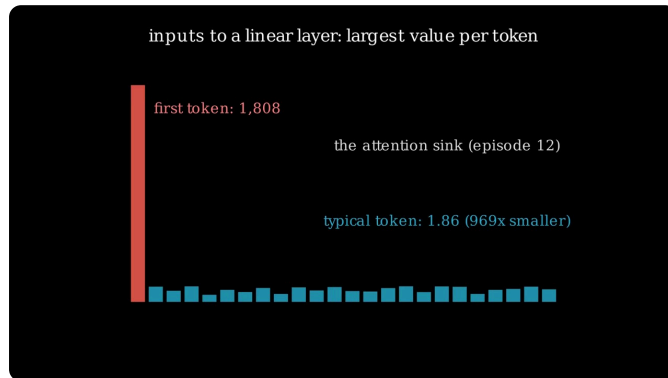
True ☐ False ☐

CONCEPT 4 OF 4

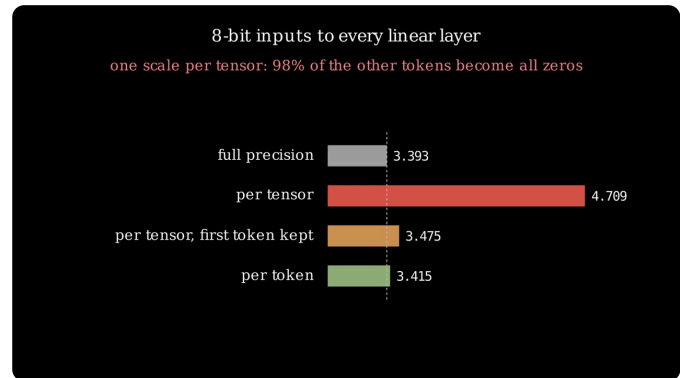
Activations: the outlier is a token

▶ Video 1:32-2:11

To use fast 8-bit arithmetic, the inputs to each layer must be rounded too. There, the outlier is a whole token: the very first one carries enormous values, the attention sink of episode 12. It has the largest input in 84 of 168 linear layers; in layer 2's down_proj it reaches 1,808, against 1.86 for a typical token.



▶ 1:52 Inputs to layer 2's down_proj: the first token reaches 1,808; a typical token 1.86 (969x smaller).



▶ 2:10 8-bit inputs to every linear layer: per tensor 4.709; first token kept 3.475; per token 3.415 (full precision 3.393).

With one scale for the whole tensor the step there is 14.2, and 98% of the other tokens round to all zeros: the loss jumps to 4.709. Keeping the first token in 16 bits brings it back to 3.475; giving every token its own scale gives 3.415, almost free against 3.393.

KEY IDEA

Find where the outliers live (rows, groups, tokens) and give them their own scale.

Check 4: Activations: the outlier is a token

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

With one 8-bit scale set by the value 1,808 ($-127..127$), what is the step, and what does a token whose largest value is 1.86 become?

4.2 MULTIPLE CHOICE

Why does one scale per token work so well here?

- A. It uses more bits
- B. Each token's step is set by its own largest value, so the first token cannot crush the others
- C. It skips the first token
- D. It rounds weights, not inputs

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Outliers set the scale · rewatch ▶ 0:08-0:35

1.1 Step 0.05; every weight with $|w| < 0.025$. $0.35 / 7 = 0.05$; values under half a step round to 0.

1.2 7 (-3..3). $2^2 - 1 = 3$ on each side, plus zero.

2. Groups and their cost · rewatch ▶ 0:35-1:12

2.1 About 3.9 GB; 14 GB. $7e9 \times 4.5 / 8 = 3.94e9$ bytes; $7e9 \times 2 = 14e9$ bytes.

2.2 One scale: only 0.9 survives, the other seven become 0. Groups of 4: the second group keeps all its weights (0.0114, -0.0086, 0.0200, -0.0114); the first still loses its three small ones. The outlier only spoils the group it is in.

3. A better grid: NF4 · rewatch ▶ 1:12-1:32

3.1 False. Both use 4 bits (16 levels); NF4 places them better for bell-shaped weights.

4. Activations: the outlier is a token · rewatch ▶ 1:32-2:11

4.1 Step 14.2; all zeros. $1,808 / 127 = 14.2$; every value below 7.1 rounds to 0.

4.2 B. The outlier is confined to one token, so a per-token scale isolates it.